

Software Architecture and Design Specification

Project: A Snake Game

Version: 1.0

Authors: Thejaswi S, Vishnu Maddirala, V S Shashidharan, Vishwas Hosamath (Team 6)

Date: 09-09-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	09-09-2026	Thejaswi S, Vishnu Maddirala, V S Shashidharan, Vishwas Hosamath	Initial draft architecture and design specification created from the Snake Game SRS v0.1

Approvals

Role	Name	Signature/Date
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of A Snake Game system.

1.2 Scope

Covers Snake Game user services: Classic Mode, Obstacle Mode, Power-Up Mode, Two-Player Battle Mode, Survival Mode, along with player registration and authentication, player profiles, scoring, and leaderboard management.

1.3 Audience

Software developers, software testers, project team members, course faculty, project evaluators, system administrators, and end users/players.

1.4 Definitions

SRS, UI, FR, NFR, RTM, UML, FPS, Admin, Player, Game Session, Power-Up, Leaderboard, Collision.

2. Document Overview

2.1 How to use this document

Provides architectural deliverables including UML diagrams, component descriptions, and module design derived from the Snake Game SRS.

2.2 Related Documents

SRS (A Snake Game SRS v0.1), RTM (Requirements Traceability Matrix)

3. Architecture

3.1 Goals & Constraints

Goals: modular, maintainable, responsive (movement input processed within 100 ms; at least 30 FPS during normal single-player gameplay), and secure (password hashing, restricted administrator functions).

Constraints: playable using standard keyboard controls; Windows-based desktop/laptop systems; display resolution of at least 1280 x 720; local database or configured application database;

3.2 Stakeholders & Concerns

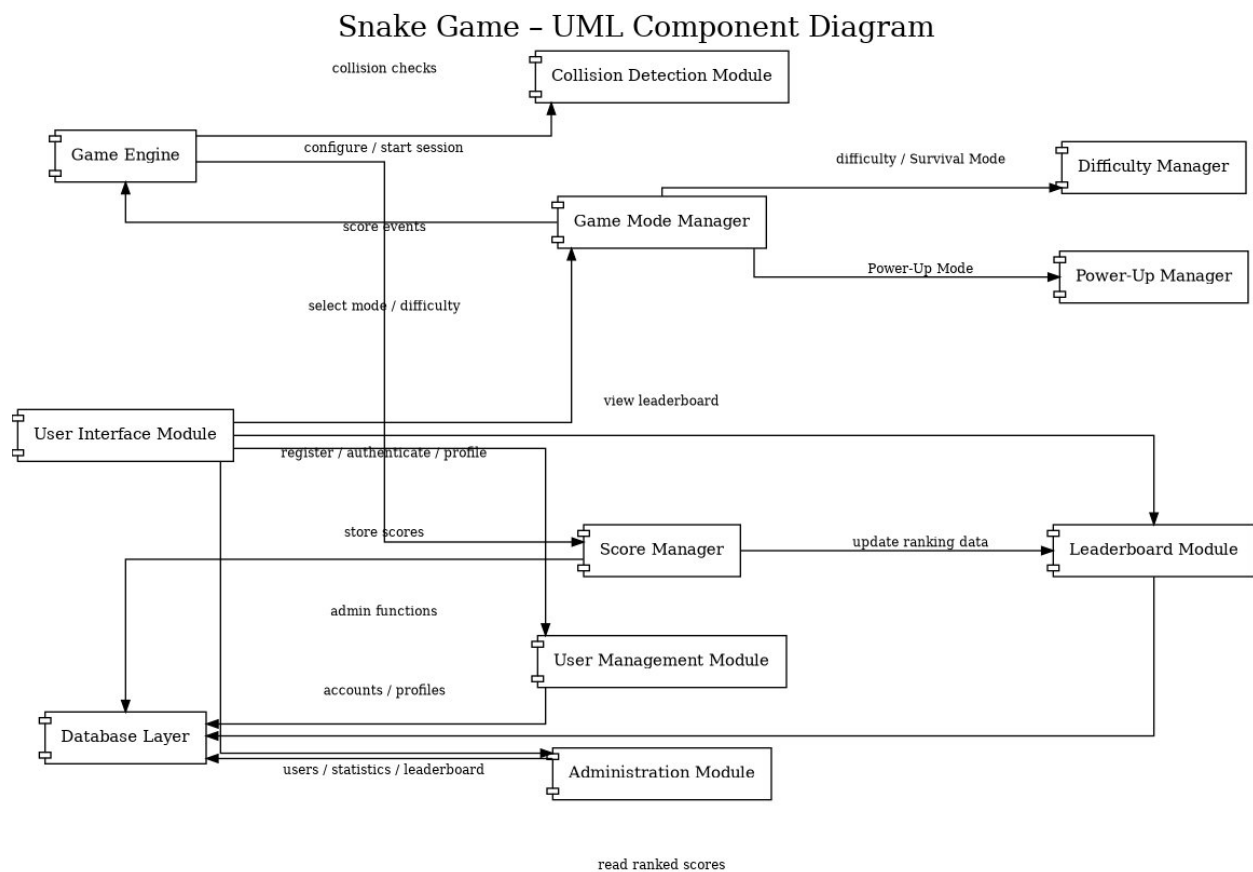
Player: usability, responsive controls, clear interface understandable to new players.

Administrator: manageability of users, statistics, leaderboard data, and selected parameters.

Course Faculty/Project Evaluators: correctness, reliability, and completeness of implemented requirements.

Developers: modularity, independent development, testing, and maintenance of components.

3.3 Component (UML) Diagram



3.4 Component Descriptions

- User Interface Module: handles player and administrator input/output across menus, game board, score display, pause menu, game-over screen, leaderboard, player profile, settings, and administrator interface.
- Game Engine: handles real-time gameplay mechanics such as snake movement, food generation, collision detection, scoring, and game-state management.
- Game Mode Manager: manages selection and rules for Classic, Obstacle, Power-Up, Two-Player Battle, and Survival modes.
- Collision Detection Module: detects collisions between the snake and walls, its own body, obstacles, or an opponent snake where applicable.
- Score Manager: calculates and stores player scores based on applicable game rules and gameplay events.
- User Management Module: manages player registration, authentication, and profile information.
- Leaderboard Module: maintains and displays a ranked list of players based on scores.
- Database Layer: provides local database or configured application database storage for accounts, scores, and leaderboard data.
- Administration Module: provides authorized administrator functions such as viewing users and statistics, managing accounts, reviewing leaderboard data, and configuring selected parameters.

3.5 Chosen Architecture Pattern and Rationale

A modular architecture is chosen so that individual game modes and supporting components (Game Engine, Game Mode Manager, Collision Detection Module, Score Manager, Power-Up Manager, Difficulty Manager, User Management Module, Leaderboard Module, Database Layer, and Administration Module) can be developed, tested, and maintained independently. A tightly-coupled, non-modular design is rejected as it would hinder independent testing and maintenance of the five game modes and supporting services.

3.6 Technology Stack & Data Stores

Operating Environment: Windows-based desktop/laptop systems, with keyboard input and a display resolution of at least 1280 x 720.

Data Store: a local database or a configured application database is used to store player accounts, profiles, scores, and leaderboard information.

Runtime: a supported modern runtime/browser environment depending on implementation technology.

3.7 Risks & Mitigations

Risk: Inconsistent gameplay behavior across supported environments → Mitigation: consistent handling of game-state transitions and compatibility testing across the supported operating environment.

Risk: Unauthorized modification of stored scores or leaderboard records → Mitigation: normal player interfaces are prevented from directly modifying stored scores or leaderboard records; scores are preserved without unintended modification or loss under normal operating conditions.

Risk: Unauthorized access to player or administrator accounts → Mitigation: authentication and appropriate credential handling, including secure one-way password hashing.

3.8 Traceability to Requirements

Req ID	Requirement	Module
SNAKE-F-001	Player Registration	User Management Module
SNAKE-F-002	Player Authentication	Authentication Module
SNAKE-F-009	Snake Movement	Snake Controller
SNAKE-F-012	Collision Detection	Collision Manager
SNAKE-F-013	Score Calculation	Score Manager
SNAKE-F-016	Two-Player Battle	Multiplayer Module
SNAKE-F-020	Leaderboard	Leaderboard Module
SNAKE-SR-002	Restrict administrator functions to authorized accounts	Administration Module

3.9 Security Architecture

Account Protection: the system shall protect player and administrator accounts from unauthorized access through authentication and appropriate credential handling.

Data Integrity: the system shall protect player scores, leaderboard information, and other persistent game data from unauthorized modification.

- SNAKE-SR-001: passwords are stored using a secure one-way password hashing mechanism; plaintext passwords are not stored.
- SNAKE-SR-002: administrator functions are restricted to authenticated and authorized administrator accounts.
- SNAKE-SR-003: user-provided input is validated before processing or storing it.
- SNAKE-SR-004: normal player interfaces are prevented from directly modifying stored scores or leaderboard records.
- SNAKE-SR-005: authenticated sessions are terminated according to the configured session-management policy, preventing access to protected functionality after logout.

4. Design

4.1 Design Overview

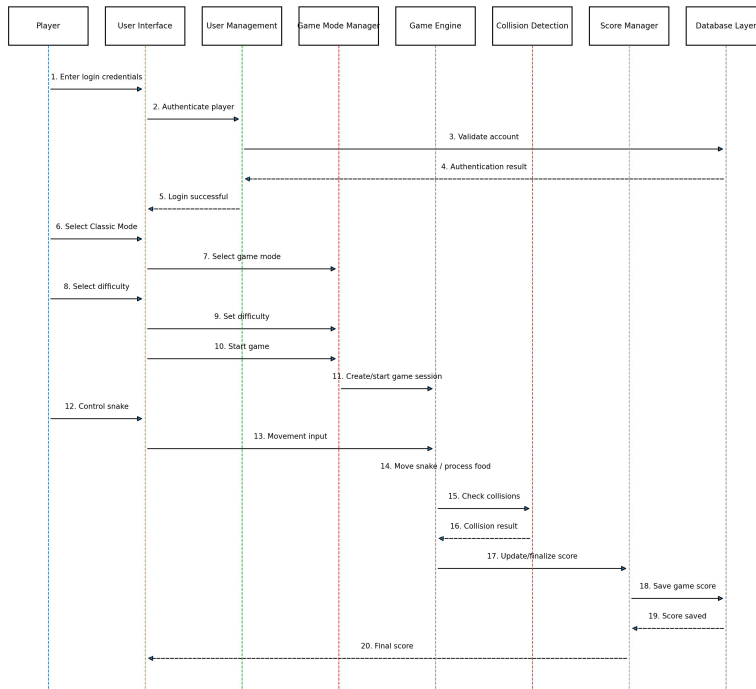
A Snake Game is designed with modular components (User Interface Module, Game Engine, Game Mode Manager, Collision Detection Module, Score Manager, Power-Up Manager, Difficulty Manager, User Management Module, Leaderboard Module, Database Layer, Administration Module) for separation of concerns and independent testing and maintenance.

4.2 UML Sequence Diagrams

At least 2 sequence diagrams covering 2 flows specific to your project:

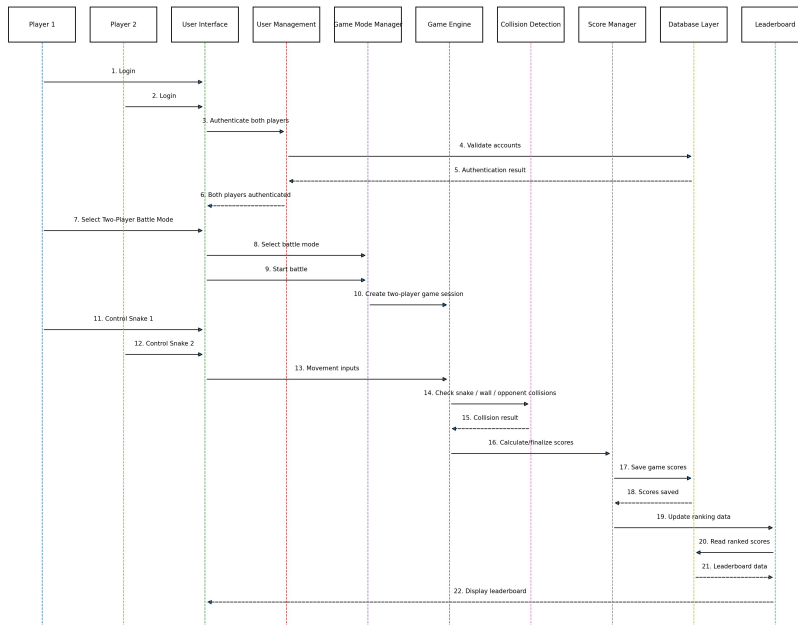
- Flow 1: Start Game and Play Classic Mode (Login → Select Game Mode → Select Difficulty → Start Game → Control Snake → Consume Food → Collision Detection → Save Game Score).

Sequence Diagram 1 – Start Game and Play Classic Mode



- Flow 2: Two-Player Battle Mode (Login for both players → Select Two-Player Battle Mode → Control Snake for Player 1 and Player 2 → Collision Detection between snakes → Score Calculation → Save Game Score → View Leaderboard).

Sequence Diagram 2 – Two-Player Battle Mode



4.3 API Design

Interface definition for at least 2 components.

Score Manager

Endpoint: /saveGameScore

Method: POST

Request: {playerId, gameMode, difficulty, finalScore}

Response: {status, savedScore}

Errors: 401 Unauthenticated Player, 400 Invalid Score Data

Game Engine / Game Mode Manager

Endpoint: /startGame

Method: POST

Request: {playerId, gameMode, difficulty}

Response: {status, gameSessionId}

Errors: 400 Invalid Game Mode or Difficulty, 401 Unauthenticated Player

4.4 Error Handling, Logging & Monitoring

Standardized error handling for invalid input, unauthenticated access, and invalid game mode/difficulty selection, consistent with input validation (SNAKE-SR-003).

No sensitive information (such as plaintext passwords) is recorded in logs, consistent with secure one-way password hashing (SNAKE-SR-001).

Monitoring includes movement input response time (target ≤ 100 ms) and frame rate during normal single-player gameplay (target ≥ 30 FPS).

4.5 UX Design

User interface screens: Login/Register screen, Main menu, Game mode selection, Difficulty selection, Game board, Score display, Pause menu, Game-over screen, Leaderboard, Player profile, Settings, and Administrator interface.

During gameplay, the interface displays the current score, game mode, difficulty, snake, food, and applicable obstacles or power-ups.

A new player shall be able to start a Classic Mode game within three primary menu interactions after successful login.

4.6 Open Issues & Next Steps

Future enhancements: online multiplayer, AI-controlled snakes, custom map creation, additional power-ups, tournament mode, cloud synchronization, achievements and badges, replay system, mobile version, and additional visual themes.

5. Appendices

5.1 Glossary

SRS: Software Requirements Specification. UI: User Interface. FR: Functional Requirement. NFR: Non-Functional Requirement. RTM: Requirements Traceability Matrix. UML: Unified Modeling Language. FPS: Frames Per Second. Admin: System Administrator. Player: A user who plays the game. Game Session: A single instance of gameplay. Power-Up: A temporary item providing a special ability or effect. Leaderboard: A ranked list of players based on scores. Collision: Contact between the snake and an invalid game object.

5.2 References

A Snake Game — Software Requirements Specification, Version 0.1, Team 6 (09-09-2026).

Course SRS template.

5.3 Tools

To be determined.